

ZÁVĚREČNÁ STUDIJNÍ PRÁCE

dokumentace



Kooperativní 2D hra v Pygame se síťovou komunikací

Autor: Matěj Fojtík
Obor: 18-20-M/01 INFORMAČNÍ TECHNOLOGIE
se zaměřením na počítačové sítě a programování
Třída: IT4
Školní rok: 2025/26

Poděkování

Prostor k poděkování (například vedoucímu práce).

Prohlášení

Prohlašuji, že jsem závěrečnou práci vypracoval samostatně a uvedl veškeré použité informační zdroje.

Souhlasím, aby tato studijní práce byla použita k výukovým a prezentačním účelům na Střední průmyslové a umělecké škole v Opavě, Praskova 399/8.

V Opavě 1. 1. 2024

.....
Podpis autora

Abstrakt

Tato práce popisuje návrh a implementaci kooperativní 2D hry vytvořené v jazyce Python s využitím knihovny Pygame. Projekt je rozšířen o síťovou vrstvu umožňující připojení více hráčů a synchronizaci jejich pozic v reálném čase. V práci jsou shrnuty požadavky na aplikaci, zvolená architektura a rozdělení projektu do modulů. Dále je popsán způsob vykreslování herní scény s kamerovým posunem, práce se sprity a základní herní smyčka. Samostatná část je věnována síťové komunikaci přes Socket.IO a problémům kompatibility knihoven (async režimy, eventlet) v prostředí Windows a Python 3.13. Závěr práce shrnuje dosažené výsledky a navrhuje možná rozšíření, například chat, lobby, synchronizaci animací a stavů hráčů a zlepšení odolnosti proti výpadkům spojení.

Klíčová slova

Python, Pygame, multiplayer, Socket.IO, klient-server, sprite

Obsah

Úvod	2
1 Popis projektu	3
1.1 Cíl práce	3
1.2 Požadavky na projekt	3
1.3 Použité technologie	4
1.4 Způsob spuštění projektu	5
2 Základní herní mechaniky	6
2.1 Architektura projektu	6
2.2 Hráčská postava	7
2.3 Systém sbírání předmětů	10
3 Databáze hráčů	15
3.1 Volba databáze	15
3.2 Datový model	15
3.3 Inicializace databáze	16
3.4 Načítání a ukládání postupu	16
3.5 Rozšiřitelnost řešení	17
4 Socket.IO	18
4.1 Proč Socket.IO	18
4.2 Architektura klient-server	18
4.3 Použité události	19
4.4 Spuštění serveru	22
4.5 Poznámky k synchronizaci a omezením	22
5 Aplikace Tiled	23
5.1 Co je Tiled	23
5.2 Využití Tiled v projektu	23
5.3 Tvorba mapy pomocí tilesetu	24
5.4 Výhody použití Tiled	25
6 NPC	26
6.1 Vytvoření NPC	26
6.2 Dialog s NPC	27

ÚVOD

Tento projekt jsem si zvolil ze dvou hlavních důvodů. Rád hraji počítačové hry a zároveň mě baví jejich programování. Vývoji her se věnuji nějakou dobu a během této doby jsem vytvořil několik vlastních projektů – od jednoduchých platformových her až po tuto složitější kooperativní hru.

Tato práce se zaměřuje na návrh a implementaci 2D počítačové hry vytvořené v programovacím jazyce Python s využitím knihovny Pygame. Hra je koncipována jako kooperativní projekt, který umožňuje propojení více hráčů prostřednictvím síťové komunikace. Součástí projektu je herní logika, práce se sprity, kolizní systém, dialogový systém, uživatelské rozhraní a základní multiplayerová funkcionality.

Důraz je kladen nejen na samotnou funkčnost aplikace, ale také na její strukturu, přehlednost zdrojového kódu a možnost dalšího rozšiřování. Projekt je rozdělen do samostatných modulů, což usnadňuje orientaci v kódu a odpovídá běžným zásadám vývoje softwaru.

Součástí práce je rovněž podrobná dokumentace, která popisuje jednotlivé fáze vývoje, použité technologie a principy implementace. Dokumentace slouží jak k vysvětlení funkce výsledné aplikace, tak jako důkaz porozumění použitým postupům a nástrojům.

1 POPIS PROJEKTU

1.1 CÍL PRÁCE

Cílem této práce bylo navrhnout a implementovat 2D kooperativní hru v jazyce Python s využitím knihovny Pygame. Základní myšlenkou projektu je vytvořit jednoduchý herní svět, ve kterém se hráč může pohybovat, interagovat s objekty a (v rozšířené verzi) spolupracovat s dalším hráčem přes síť.

Projekt je od začátku navržen modulárně, tedy rozdělen do více samostatných souborů a tříd, aby byl přehledný, snadno udržovatelný a bylo možné ho postupně rozšiřovat. To umožňuje oddělit například logiku hráče, vykreslování světa, dialogy, menu a síťovou komunikaci, což je důležité hlavně u větších projektů.

Důležitou součástí cíle bylo doplnění síťové komunikace pro režim dvou hráčů. V kooperativním režimu má být možné spustit jedno zařízení jako host (server) a druhé jako klient, přičemž mezi nimi dochází alespoň k základní synchronizaci (typicky pozice hráčů). Cílem nebylo vytvořit plnohodnotný MMO systém, ale funkční základ pro multiplayer, na který lze navázat dalšími prvky (interakce, animace, stavové informace, chat, lobby apod.).

1.2 POŽADAVKY NA PROJEKT

Před samotnou implementací byly stanoveny funkční a nefunkční požadavky. Ty sloužily jako vodítko pro vývoj, aby bylo jasné, co má výsledná aplikace splňovat, a zároveň aby bylo možné ověřit, že projekt odpovídá zadání.

1.2.1 Funkční požadavky

Funkční požadavky popisují, co má aplikace umět z pohledu uživatele (hráče) a z pohledu herní logiky:

- Aplikace umožní spustit hru v režimu jednoho hráče bez síťové komunikace, aby bylo možné otestovat logiku a stabilitu hry i bez připojení.

- Hra bude obsahovat základní herní mechaniky: pohyb hráče, kolize s překážkami a vykreslování herního světa pomocí spritů.
- Aplikace umožní spustit hosta (server) a připojit klienta (druhého hráče), tedy vytvořit jednoduchý multiplayer režim.
- V multiplayer režimu bude synchronizována alespoň pozice hráčů tak, aby oba hráči viděli pohyb toho druhého v reálném čase.

1.2.2 Nefunkční požadavky

Nefunkční požadavky popisují vlastnosti projektu z hlediska kvality, použitelnosti a technických omezení. V této práci byly klíčové hlavně tyto body:

- Projekt bude rozdělen do více modulů pro lepší čitelnost a údržbu, aby nebyla celá logika soustředěna do jednoho souboru.
- Hra pobeží plynule na běžných školních počítačích, typicky kolem 60 FPS, což je důležité pro pocit ovladatelnosti.
- Síťová část bude navržena tak, aby byla použitelná na Windows a v prostředí Pythonu 3.13, kde mohou některé knihovny a async režimy způsobovat problémy.

1.3 POUŽITÉ TECHNOLOGIE

Při vývoji projektu byly použity následující technologie a knihovny:

- **Python** – hlavní programovací jazyk. Použit pro herní logiku, správu objektů i síťovou komunikaci.
- **pygame-ce** – knihovna pro tvorbu 2D her. Zajišťuje práci s oknem, událostmi (klávesnice), vykreslováním, sprity a časováním smyčky.
- **pygame-gui** – knihovna pro tvorbu grafického uživatelského rozhraní (např. menu, nastavení, pauza). Díky tomu není potřeba ručně kreslit tlačítka a řešit klikání „od nuly“.
- **python-socketio** – knihovna pro síťovou komunikaci. Umožňuje jednodušší výměnu zpráv mezi klientem a serverem formou událostí.
- **pytmx** – knihovna pro načítání map vytvořených v editoru Tiled (pokud je mapový systém v projektu použit). Usnadňuje práci s tilemapou a kolizními vrstvami.

1.4 ZPŮSOB SPUŠTĚNÍ PROJEKTU

Projekt se spouští pomocí souboru `main.py`, který inicializuje Pygame a spustí hlavní herní smyčku. Před prvním spuštěním je nutné nainstalovat závislosti ze souboru `requirements.txt`. Doporučený postup je použití virtuálního prostředí (venv), aby se instalované knihovny nepletly s ostatními projekty.

```
1 git clone https://github.com/MFrly/zaverecny-projekt.git
2 cd <nazev-repozitare>
3 python -m venv venv
4
5 venv\Scripts\activate    # Windows
6 # nebo:
7 source venv/bin/activate # Linux
8
9 pip install -r requirements.txt
10 python code/main.py
```

Kód 1.1: Instalace závislostí a spuštění projektu

V závislosti na implementaci menu lze hru spustit v režimu jednoho hráče nebo zvolit síťový režim (host / klient). Pokud je síťová část řešena přes Socket.IO, je typicky potřeba, aby hostitel spustil server jako první a klient se následně připojil pomocí IP adresy a portu.

2 ZÁKLADNÍ HERNÍ MECHANIKY

Tato kapitola se zabývá základními herními mechanikami implementovanými v projektu. Popisuje strukturu zdrojového kódu, chování hráčské postavy, systém sbírání předmětů a další klíčové prvky, které tvoří funkční herní základ. Tyto mechaniky byly implementovány v první fázi vývoje projektu, ještě před přidáním síťové komunikace a pokročilejších herních prvků.

2.1 ARCHITEKTURA PROJEKTU

Projekt je navržen modulárně a jednotlivé části herní logiky jsou rozděleny do samostatných souborů. Tento přístup zvyšuje přehlednost kódu, usnadňuje údržbu a umožňuje postupné rozšiřování projektu o další funkce, jako je dialogový systém nebo síťová hra.

Zdrojové soubory hry jsou umístěny ve složce `code`, přičemž každý soubor má jasně definovanou odpovědnost. Hlavním vstupním bodem aplikace je soubor `main.py`, který inicializuje herní prostředí a řídí průběh celé hry.

Základní strukturu projektu lze rozdělit do několika logických částí:

- **Herní logika** – řízení herní smyčky, aktualizace objektů a vykreslování.
- **Entity** – hráčská postava, NPC a další herní objekty.
- **Sprity a kolize** – grafická reprezentace objektů a kolizní systém.
- **Uživatelské rozhraní** – menu, pauza a dialogy.
- **Síťová komunikace** – podpora kooperativního režimu.

Herní smyčka je implementována v metodě `run()` třídy `Game` v souboru `main.py`. V každém průchodu smyčkou dochází ke zpracování událostí, aktualizaci herního stavu a vykreslení herního obrazu.

```

1 while self.running:
2     dt = self.clock.tick(60) / 1000
3
4     for event in pygame.event.get():
5         # zpracování vstupu
6
7     self.all_sprites.update(dt)
8     self.all_sprites.draw(self.player.rect.center)
9
10    pygame.display.update()

```

Kód 2.1: Zjednodušená struktura herní smyčky

Pro správu herních objektů je využíván sprite systém knihovny Pygame. Jednotlivé sprity jsou seskupeny do skupin, které umožňují hromadnou aktualizaci a vykreslování objektů. Speciální skupina `AllSprites` navíc implementuje jednoduchý kamerový systém pomocí posunu vykreslování.

Takto navržená architektura tvoří stabilní základ, na který bylo možné postupně navázat další herní mechaniky, jako jsou dialogy s NPC, sbírání předmětů nebo síťová synchronizace hráčů.

2.2 HRÁČSKÁ POSTAVA

Hráčská postava představuje hlavní entitu, prostřednictvím které uživatel ovládá hru. Její implementace se nachází v souboru `player.py` a je založena na třídě `Sprite` z knihovny Pygame.

Třída `Player` dědí od třídy `pygame.sprite.Sprite`, což umožňuje snadné zařazení hráče do sprite skupin a využití vestavěných funkcí pro aktualizaci a vykreslování.

2.2.1 Inicializace hráče

Při vytvoření instance hráče se nastavuje jeho počáteční pozice, grafická reprezentace, kolizní oblast a základní vlastnosti jako rychlost pohybu nebo směr.

Kód 2.2: Inicializace hráčské postavy

```
class Player(pygame.sprite.Sprite):
    def __init__(self, pos, groups, collision_sprites, name="Hráč"):
        super().__init__(groups)
        self.image = pygame.image.load(
            join('images', 'player', 'down', '0.png')
        ).convert_alpha()
        self.rect = self.image.get_rect(center=pos)
        self.hitbox_rect = self.rect.inflate(-60, -60)
```



Obrázek 2.1: Hráčská postava

Pro účely kolizí je použita samostatná kolizní oblast (`hitbox_rect`), která je menší než samotný sprite. Tento přístup zlepšuje plynulost pohybu a zabraňuje nechtěným kolizím s okolními objekty.

2.2.2 Ovládání hráče

Ovládání hráče je realizováno pomocí klávesnice. Stav kláves je zjišťován pomocí funkce `pygame.key.get_pressed()` a výsledný směr pohybu je uložen do vektoru.

```

1 def input(self):
2     keys = pygame.key.get_pressed()
3     self.direction.x = int(keys[pygame.K_RIGHT]) - int(keys[pygame.K_LEFT])
4     self.direction.y = int(keys[pygame.K_DOWN]) - int(keys[pygame.K_UP])
5
6     if self.direction.length_squared() > 0:
7         self.direction = self.direction.normalize()

```

Kód 2.3: Zpracování vstupu hráče

Normalizace vektoru zajišťuje, že rychlost pohybu hráče zůstává stejná ve všech směrech, včetně diagonálního pohybu.

2.2.3 Pohyb a kolize

Pohyb hráče je rozdělen na horizontální a vertikální složku. Tento přístup umožňuje přesnější řešení kolizí s okolními objekty herního světa.

```

1 def move(self, dt):
2     self.hitbox_rect.x += self.direction.x * self.speed * dt
3     self.collision('horizontal')
4
5     self.hitbox_rect.y += self.direction.y * self.speed * dt
6     self.collision('vertical')
7
8     self.rect.center = self.hitbox_rect.center

```

Kód 2.4: Pohyb a kolizní detekce

Kolizní kontrola probíhá vůči skupině kolizních objektů, které jsou načteny z mapy vytvořené v editoru Tiled. Při detekci kolize je pozice hráče upravena tak, aby nedošlo k průniku objektů.

2.2.4 Aktualizace hráče

Metoda `update()` je volána v každém snímku herní smyčky. Zajišťuje zpracování vstupu a aktualizaci pozice hráče, pokud je povoleno ovládání (například není otevřen dialog nebo pauza).

```
1 def update(self, dt):  
2     if self.input_enabled:  
3         self.input()  
4         self.move(dt)
```

Kód 2.5: Aktualizační metoda hráče

Tímto způsobem je hráčská postava pevně integrována do herní smyčky a reaguje plynule na vstupy uživatele. Implementace zároveň umožňuje dočasně zablokovat ovládání hráče, například během dialogu s NPC nebo při otevřeném menu.

2.3 SYSTÉM SBÍRÁNÍ PŘEDMĚTŮ

Součástí herních mechanik je systém sbírání předmětů, který je v projektu využit pro získávání jednotlivých částí klíče potřebného k postupu do další části hry. Tento systém je navržen modulárně tak, aby bylo možné v budoucnu snadno přidávat další typy sbíratelných objektů.

2.3.1 Reprezentace předmětů

Jednotlivé části klíče jsou reprezentovány třídou `KeyPart`, která se nachází v souboru `sprites.py`. Tato třída dědí od `pygame.sprite.Sprite` a obsahuje informace o grafickém vzhledu předmětu a jeho identifikátoru.

```
1 class KeyPart(pygame.sprite.Sprite):
2     def __init__(self, pos, part_id, surf, groups):
3         super().__init__(groups)
4         self.image = surf
5         self.rect = self.image.get_rect(topleft=pos)
6         self.hitbox_rect = self.rect.inflate(-8, -8)
7         self.part_id = part_id
```

Kód 2.6: Třída `KeyPart`

Každá část klíče má vlastní identifikátor `part_id`, díky kterému je možné rozlišovat jednotlivé části a zabránit jejich duplicitnímu sběru.

2.3.2 Umístění předmětů do herního světa

Předměty jsou do herního světa vkládány při inicializaci mapy. Konkrétní pozice klíčů jsou v této fázi vývoje definovány pevně v kódu, což zjednodušuje testování herních mechanik.

```
1 def spawn_keys(self):
2     KeyPart((300, 400), 1, self.key_img, [self.all_sprites, self.item_sprites])
3     KeyPart((900, 250), 2, self.key_img, [self.all_sprites, self.item_sprites])
4     KeyPart((1200, 800), 3, self.key_img, [self.all_sprites, self.item_sprites])
```

Kód 2.7: Vytvoření částí klíče

Předměty jsou zařazeny do samostatné skupiny `item_sprites`, což umožňuje jejich snadnou správu a kolizní detekci.

2.3.3 Detekce sebrání předmětu

Sebrání předmětu je realizováno pomocí kolizní detekce mezi hitboxem hráče a hitboxem předmětu. Pokud dojde ke kolizi, předmět je odstraněn ze hry a jeho identifikátor je uložen do inventáře hráče.

```

1 def check_key_pickup(self):
2     for key in list(self.item_sprites):
3         if self.player.hitbox_rect.colliderect(key.hitbox_rect):
4             self.player.key_parts.add(key.part_id)
5             key.kill()

```

Kód 2.8: Kontrola sebrání klíče

Inventář hráče je v tomto projektu reprezentován množinou (*set*), která zajišťuje, že každá část klíče může být uložena pouze jednou.

2.3.4 Vazba na herní postup

Sebrané části klíče přímo ovlivňují průběh hry. Jejich počet je kontrolován při interakci s nehratelnou postavou (NPC), která hráči poskytuje zpětnou vazbu a v případě získání všech částí složí finální klíč.

Tento přístup propojuje sbírání předmětů s dialogovým systémem a vytváří jasnou herní motivaci pro průzkum mapy.

2.3.5 Herní smyčka a aktualizací cyklus

Základním stavebním prvkem celé hry je herní smyčka (angl. *game loop*), která zajišťuje plynulý běh aplikace a pravidelnou aktualizaci herního stavu. Herní smyčka se opakuje po dobu běhu programu a v každém průchodu vykonává několik klíčových kroků.

V projektu je herní smyčka implementována v metodě `run()` třídy `Game` v souboru `main.py`. Její struktura odpovídá běžnému návrhu herních aplikací.

Zpracování vstupu

První část herní smyčky se věnuje zpracování vstupu od uživatele. Pomocí systému událostí knihovny Pygame jsou zachyceny události klávesnice, myši a také systémové události, jako je zavření okna.

```

1 for event in pygame.event.get():
2     if event.type == pygame.QUIT:
3         self.running = False
4
5     if event.type == pygame.KEYDOWN:
6         if event.key == pygame.K_ESCAPE:
7             self.pause.open()

```

Kód 2.9: Zpracování událostí

Tímto způsobem je možné reagovat na akce hráče (spuštění dialogu, otevření pauzy, ukončení hry) bez blokování běhu aplikace.

Aktualizace herního stavu

Po zpracování vstupu následuje aktualizace herního stavu. V této fázi dochází k pohybu hráče, kontrole kolizí, sbírání předmětů a synchronizaci se síťovou částí hry.

Aktualizace je řízena časovým krokem *dt* (delta time), který zajišťuje nezávislost pohybu na snímkové frekvenci.

```

1 dt = self.clock.tick(60) / 1000
2
3 if not self.pause.is_open() and not self.dialog.is_active():
4     self.all_sprites.update(dt)
5     self.check_key_pickup()

```

Kód 2.10: Aktualizační část smyčky

Použití *dt* umožňuje, aby hra běžela stejně rychle na různě výkonných počítačích.

Vykreslování herní scény

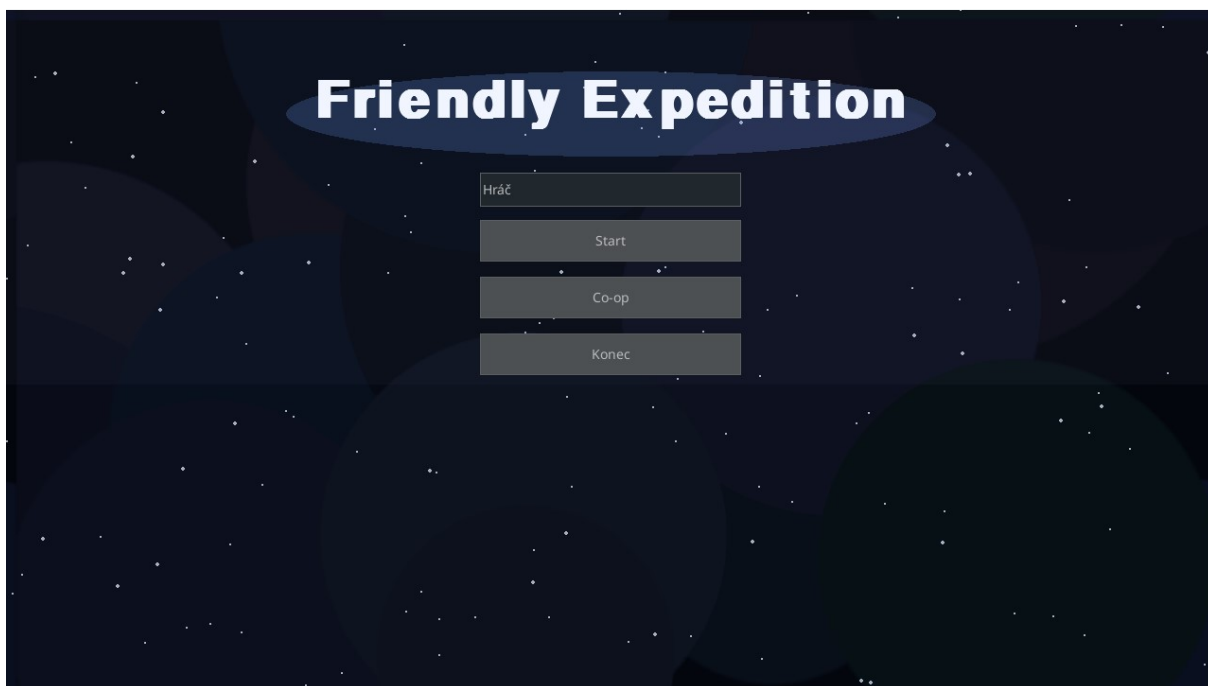
Poslední část herní smyčky se stará o vykreslení herní scény. Nejprve je obrazovka vymazána, poté jsou vykresleny všechny sprity s ohledem na aktuální pozici kamery.

```

1 self.display_surface.fill("black")
2 self.all_sprites.draw(self.player.rect.center)
3 pygame.display.update()

```

Kód 2.11: Vykreslení herní scény v hlavní smyčce



Obrázek 2.2: Hlavní menu

Kamera je realizována pomocí posunu vykreslování, díky čemuž zůstává hráč přibližně uprostřed obrazovky, zatímco herní svět se pohybuje kolem něj.

Řízení běhu hry

Herní smyčka běží po dobu, kdy je proměnná `running` nastavena na hodnotu `True`. Ukončení smyčky nastává například při zavření okna, návratu do hlavního menu nebo ukončení aplikace.

Tento přístup umožňuje snadné přepínání mezi herními stavy, jako je hlavní menu, samotná hra nebo pauza.

3 DATABÁZE HRÁČŮ

Součástí projektu je jednoduchá databázová vrstva, která slouží k ukládání a načítání postupu hráče. Vzhledem k tomu, že hra je zaměřena na kooperativní a rozšiřitelný herní svět, je vhodné uchovávat stav hráče nezávisle na běhu aplikace, aby bylo možné na hru navázat po dalším spuštění.

V této fázi projektu databáze ukládá především informace související s herním postupem (například sebrané části klíče a informaci, zda už hráč vlastní finální „master key“). Databáze je navržena modulárně tak, aby bylo možné v budoucnu jednoduše přidat další položky například inventář, zkušenosti, úroveň hráče, poslední pozici nebo nastavení hry.

3.1 VOLBA DATABÁZE

Pro ukládání dat byla zvolena databáze **SQLite**. Jedná se o lehké databázové řešení uložené v jednom souboru, které nevyžaduje samostatný server a je velmi vhodné pro menší aplikace, školní projekty a singleplayer/kooperativní hry.

Výhody SQLite v kontextu tohoto projektu:

- jednoduchá instalace a použití (je dostupná v Pythonu přes modul `sqlite3`),
- uložení dat do jednoho souboru (snadná přenositelnost),
- dostatečný výkon pro malé množství dat,
- možnost rozšíření struktury (přidávání sloupců/tabulek).

3.2 DATOVÝ MODEL

Databáze pracuje s tabulkou hráčů, kde je každý hráč identifikován svým jménem (v projektu se jméno vybírá v menu a předává se do třídy `Game`). Jméno hráče tak funguje jako primární klíč, díky čemuž lze snadno načíst správný záznam.

Ukládané hodnoty:

- `name` – jméno hráče (identifikace profilu),
- `key_parts` – seznam získaných částí klíče,
- `has_master_key` – logická hodnota určující, zda hráč již vlastní finální klíč.

Protože Python používá pro části klíče datovou strukturu `set`, která se přímo neukládá do SQL, je množina před uložením převedena na serializovaný formát (např. JSON seznam). Při načítání je tento seznam opět převeden zpět na `set`, čímž je zachována vlastnost, že každá část klíče může existovat pouze jednou.

3.3 INICIALIZACE DATABÁZE

Databáze se inicializuje při startu hry. Při inicializaci se vytvoří soubor databáze (pokud ještě neexistuje) a následně tabulka hráčů.

Inicializace je volána v konstruktoru třídy `Game`:

```
1 # DB init
2 init_db()
```

Kód 3.1: Inicializace databáze při startu hry

Tento přístup zajišťuje, že hra je schopná databázi použít bez ohledu na to, zda je spuštěna poprvé nebo jde o opakované spuštění.

3.4 NAČÍTÁNÍ A UKLÁDÁNÍ POSTUPU

Načtení dat probíhá při vytvoření hráče (po spawnu v mapě) a před samotným spuštěním hry. Pokud databáze obsahuje záznam se stejným jménem hráče, hra převezme uložené hodnoty a nastaví je do instance hráče.

```
1 def load_player_progress(self):
2     data = load_player(self.player_name)
3     if data:
4         self.player.key_parts = data["key_parts"]
5         self.player.has_master_key = data["has_master_key"]
```

Kód 3.2: Načítání postupu hráče z databáze

Ukládání probíhá při ukončení hry, při návratu do menu a také jako pojistka při zavření okna. Tím je minimalizováno riziko ztráty postupu při neočekávaném ukončení aplikace.

```
1 def save_player_progress(self):  
2     save_player(self.player_name, self.player.key_parts, self.player.has_master_key)
```

Kód 3.3: Ukládání postupu hráče do databáze

V samotné herní smyčce je ukládání vyvoláno například při události `pygame.QUIT` nebo při volbě menu/quit v pauze:

```
1 if event.type == pygame.QUIT:  
2     self.save_player_progress()  
3     self.running = False
```

Kód 3.4: Uložení při ukončení hry

3.5 ROZŠÍŘITELNOST ŘEŠENÍ

Návrh databázové vrstvy je záměrně jednoduchý, ale připravený pro budoucí rozšíření. V další verzi projektu je možné ukládat například:

- poslední pozici hráče a aktuální mapu,
- inventář a vybavení,
- zkušenosti, úroveň a herní statistiky,
- nastavení hry (hlasitost, rozlišení),
- stav splněných úkolů a dialogů.

V případě rozšíření kooperativního režimu lze databázi využít i pro ukládání informací o serveru/lobby nebo pro uchování historie hráčů, nicméně pro plnohodnotný multiplayer by bylo vhodnější uvažovat serverovou databázi a ukládání stavu na straně hostitele.

4 SOCKET.IO

V této kapitole je popsána síťová část projektu, která umožňuje kooperativní hraní více hráčů. Pro realizaci síťové komunikace byla zvolena knihovna `python-socketio`, která poskytuje událostmi řízené rozhraní a umožňuje obousměrnou komunikaci v reálném čase.

Síťová vrstva je navržena tak, aby byla v projektu snadno použitelná a zároveň byla oddělena od hlavní herní logiky. Díky tomu lze kooperativní režim zapnout pouze v případě potřeby (host / klient) a singleplayer režim zůstává funkční i bez běhu serveru.

4.1 PROČ SOCKET.IO

Socket.IO zjednodušuje výměnu dat mezi klientem a serverem pomocí pojmenovaných událostí. V porovnání s čistými TCP/UDP sokety poskytuje:

- jednoduché definování událostí (`connect`, `disconnect`, vlastní eventy),
- automatickou správu připojení a možnost reconnectu na klientovi,
- přenos strukturovaných dat (typicky ve formátu JSON),
- snadnou rozšiřitelnost o další typy zpráv (chat, synchronizace animací apod.).

4.2 ARCHITEKTURA KLIENT–SERVER

Kooperativní režim využívá architekturu klient–server. Jeden hráč spustí server (host) a druhý hráč se připojí jako klient.

4.2.1 Server

Server je implementován třídou `CoopServer`. Pro samotný Socket.IO server je využita instance `socketio.Server` a jako WSGI aplikace slouží `socketio.WSGIApp`.

Každému připojenému klientovi je přiřazeno unikátní číselné ID, které se zvyšuje pomocí počítadla `next_id`. Server si udržuje slovník `players`, kde klíčem je `sid` (session id Socket.IO) a hodnotou je struktura s informacemi o hráči:

```

1 self.players = {}    # sid -> {"id": int, "x": .., "y": ..}
2 self.next_id = 1

```

Kód 4.1: Udržování stavu hráčů na serveru

4.2.2 Klient

Klientská část je implementována třídou `CoopClient`. Používá `socketio.Client` se zapnutým automatickým reconnectem. Přijaté síťové zprávy se neaplikují okamžitě přímo do herních objektů, ale ukládají se do fronty (`queue.Queue`). Tím je oddělena síťová vrstva od herní smyčky a minimalizuje se riziko problémů s paralelismem a přístupem k herním datům.

```

1 self.recv_queue = queue.Queue()
2
3 @self.sio.event
4 def state(data):
5     self.recv_queue.put(data)

```

Kód 4.2: Fronta pro přijaté zprávy na klientovi

4.3 POUŽITÉ UDÁLOSTI

Komunikace mezi serverem a klientem je založena na událostech. V projektu jsou použity následující eventy:

4.3.1 connect (server i klient)

Událost `connect` se aktivuje při navázání spojení. Na serveru je zde vytvořen nový hráčský záznam, přiřazeno ID a odeslána zpráva `welcome`.

```

1 @self.sio.event
2 def connect(sid, environ):
3     pid = self.next_id; self.next_id += 1
4     self.players[sid] = {"id": pid, "x": 0, "y": 0}
5     self.sio.emit("welcome", {"player_id": pid}, to=sid)
6     self._broadcast_state()

```

Kód 4.3: Server: vytvoření hráče při připojení

Na klientovi `connect` pouze potvrdí navázání spojení (logování).

4.3.2 welcome (server → klient)

Událost welcome slouží k předání ID hráče klientovi. Klient si hodnotu uloží do proměnné `player_id`, která může být následně využita například pro rozlišení lokálního hráče od vzdálených hráčů.

```
1 @self.sio.event
2 def welcome(data):
3     self.player_id = data.get("player_id")
```

Kód 4.4: Klient: přijetí ID hráče

4.3.3 move (klient → server)

Klient odesílá serveru vlastní pozici pomocí události move. Pro jednoduchost se odesílají pouze souřadnice `x` a `y`.

```
1 def send_pos(self, x, y):
2     if self.sio.connected:
3         self.sio.emit("move", {"x": int(x), "y": int(y)})
```

Kód 4.5: Klient: odeslání pozice na server

Server po přijetí zprávy aktualizuje odpovídající záznam hráče a poté rozesílá nový stav všem.

```
1 @self.sio.event
2 def move(sid, data):
3     if sid in self.players:
4         p = self.players[sid]
5         p["x"] = int(data.get("x", 0))
6         p["y"] = int(data.get("y", 0))
7         self._broadcast_state()
```

Kód 4.6: Server: zpracování pohybu a rozeslání stavu

4.3.4 state (server → klienti)

Událost state slouží k odeslání kompletního stavu všech hráčů. Server posílá seznam hráčů (ID a pozice), což umožňuje klientům aktualizovat vzdálené entity.

```

1 def _broadcast_state(self):
2     self.sio.emit("state", {"type": "state", "players": list(self.players.values())})

```

Kód 4.7: Server: broadcast kompletního stavu

Na klientovi se přijatý stav ukládá do fronty. Herní část si poté zprávy vyzvedne v rámci herní smyčky a aplikuje je na vzdálené hráče.

```

1 def get_messages(self):
2     out = []
3     while not self.recv_queue.empty():
4         out.append(self.recv_queue.get_nowait())
5     return out

```

Kód 4.8: Klient: čtení přijatých zpráv

4.3.5 disconnect

Při odpojení klienta server odstraní hráče ze slovníku a opět rozesílá stav, aby ostatní klienti mohli vzdáleného hráče odstranit ze hry.

```

1 @self.sio.event
2 def disconnect(sid):
3     if sid in self.players:
4         del self.players[sid]
5         self._broadcast_state()

```

Kód 4.9: Server: odstranění hráče při odpojení

4.4 SPUŠTĚNÍ SERVERU

Server běží ve vlastním vlákne, aby neblokoval herní smyčku. Pro WSGI server je použit eventlet, který poskytuje jednoduchý způsob, jak server spustit.

```
1 def start(self):
2     import eventlet
3     self.thread = threading.Thread(
4         target=lambda: eventlet.wsgi.server(eventlet.listen((self.host, self.port)), self.app),
5         daemon=True
6     )
7     self.thread.start()
```

Kód 4.10: Spuštění serveru v samostatném vlákne

4.5 POZNÁMKY K SYNCHRONIZACI A OMEZENÍM

Sít'ová synchronizace je v aktuální verzi projektu záměrně jednoduchá: server rozesílá kompletní stav všech hráčů při každé změně pozice. Toto řešení je snadno implementovatelné a přehledné pro školní projekt, ale pro větší počet hráčů by bylo vhodné optimalizovat:

- odesílání pouze změn (delta updates),
- omezení frekvence odesílání (tick rate),
- interpolaci pohybu na klientovi pro plynulejší zobrazení,
- řešení ztrát spojení a opětovného připojení (reconnect + resync).

Dále je vhodné zmínit, že sít'ová část pracuje s oddělením příjmu zpráv do fronty, což umožňuje bezpečné zpracování dat v herní smyčce bez přímého zásahu Socket.IO callbacků do herních objektů.

5 APLIKACE TILED

Pro tvorbu herních map byl v projektu použit externí nástroj Tiled Map Editor. Tento nástroj výrazně usnadňuje návrh rozsáhlých 2D map a odděluje vizuální podobu herního světa od samotné herní logiky.

Použití aplikace Tiled umožnilo vytvářet mapy přehledně, rychle je upravovat a zároveň je jednoduše načítat přímo do hry pomocí knihovny pytmx.

5.1 CO JE TILED

Tiled je open-source editor map určený především pro 2D hry. Umožňuje vytvářet mapy složené z dlaždic (tzv. tiles), které jsou organizovány do vrstev.

Mapa v aplikaci Tiled je tvořena několika základními prvky:

- **Tilesheet** – sada grafických dlaždic,
- **Tile layer** – vrstva obsahující dlaždice mapy,
- **Object layer** – vrstva s objekty (kolize, entity),
- **Properties** – vlastní vlastnosti objektů a vrstev.

Výhodou Tiled je možnost ukládání map do formátu .tmx, který je založen na XML a lze jej snadno zpracovat pomocí programového kódu.

5.2 VYUŽITÍ TILED V PROJEKTU

V projektu je aplikace Tiled využita pro kompletní návrh herního světa. Mapa je rozdělena do několika vrstev, z nichž každá má ve hře specifický účel.

Použité vrstvy:

- **Ground** – základní dlaždice terénu,
- **Objects** – objekty s kolizí a viditelnou grafikou,

- **Collisions** – neviditelné kolizní oblasti,
- **Entities** – herní entity (hráč, nepřítel, NPC).

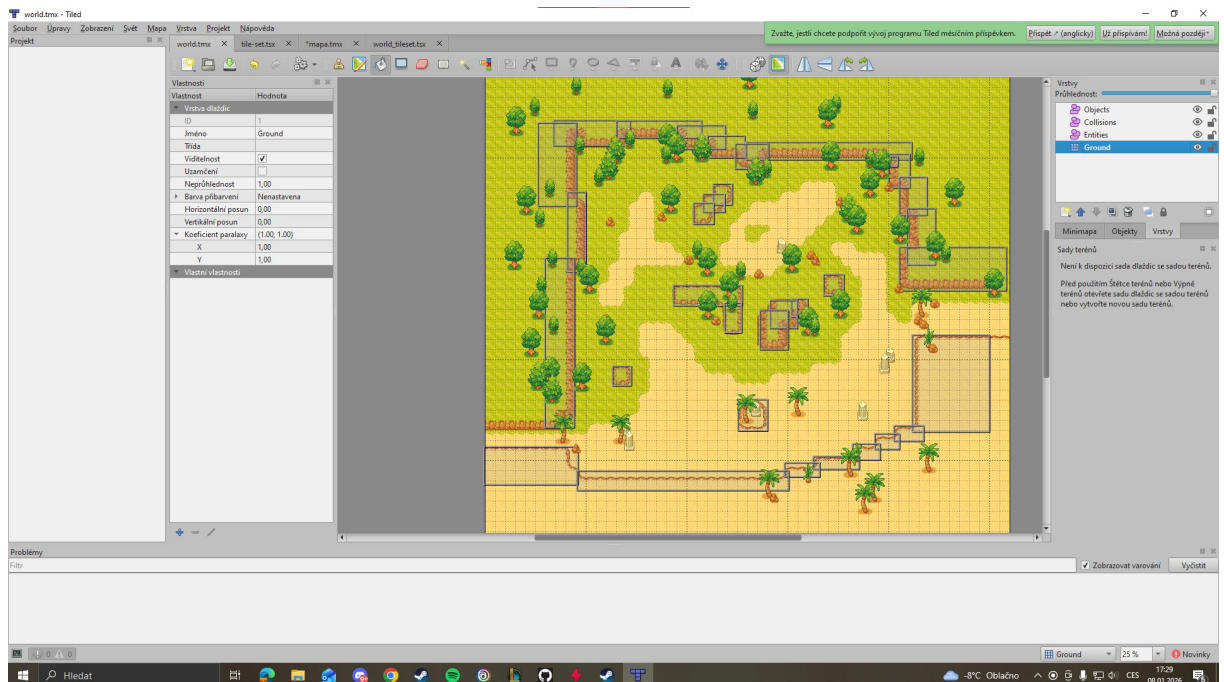
Načítání mapy do hry probíhá pomocí knihovny pytmx, která umožňuje snadno získat přístup k jednotlivým vrstvám a jejich obsahu.

Ukázka načtení mapy v projektu:

```
1 tmx = load_pygame(join("data", "maps", "world.tmx"))
```

Kód 5.1: Načtení mapy pomocí knihovny pytmx

5.3 TVORBA MAPY POMOCÍ TILESETU



Obrázek 5.1: Ukázka Tiled

Mapa je sestavena z jednoho hlavního tilesetu, který obsahuje grafiku terénu, budov, dekorací a dalších objektů. Každá dlaždice má pevně dané rozměry, které odpovídají velikosti herního pole.

V tomto projektu je velikost jedné dlaždice:

```
1 TILE_SIZE = 64
```

Kód 5.2: Velikost dlaždice

Díky tomu je možné snadno převádět souřadnice z mapy do herního světa pomocí jednoduchého násobení.

Při návrhu mapy byla zvláštní pozornost věnována:

- průchodnosti terénu,
- logickému rozmístění kolizí,
- oddělení vizuálních objektů od kolizních ploch.

Kolizní objekty jsou definovány v samostatné vrstvě, aby nebyly závislé na grafice a bylo možné je kdykoliv upravit bez nutnosti měnit vzhled mapy.

5.4 VÝHODY POUŽITÍ TILED

Použití aplikace Tiled přineslo do projektu několik zásadních výhod:

- výrazné zrychlení tvorby map,
- přehledné oddělení dat od herní logiky,
- snadnou úpravu mapy bez zásahu do zdrojového kódu,
- možnost budoucího rozšíření o další mapy.

Díky těmto vlastnostem se Tiled stal klíčovým nástrojem pro návrh herního světa v tomto projektu.

6 NPC

NPC (Non-Player Character) jsou nehráčské postavy, které ovládá hra a slouží k interakci s hráčem. V tomto projektu plní NPC především informační a dějovou roli, zejména v podobě dialogů a předávání herních úkolů.

6.1 VYTVOŘENÍ NPC

Každé NPC je v projektu implementováno jako samostatná třída, která dědí ze třídy `pygame.sprite.Sprite`. Díky tomu lze NPC jednoduše spravovat pomocí sprite skupin a zapojit je do vykreslování i kolizního systému hry.

Základní struktura třídy NPC obsahuje:

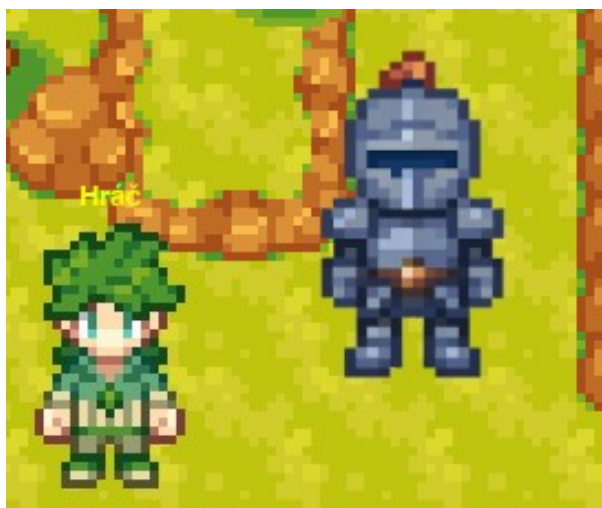
- identifikátor NPC,
- grafický vzhled postavy,
- kolizní hitbox,
- logiku aktualizace.

Ukázka třídy NPC:

```
1 class NPC(pygame.sprite.Sprite):
2     def __init__(self, pos, groups, npc_id):
3         super().__init__(groups)
4         self.npc_id = npc_id
5         self.image = pygame.image.load(
6             join('images', 'player', 'down', '0.png')
7         ).convert_alpha()
8         self.rect = self.image.get_rect(center=pos)
9         self.hitbox_rect = self.rect.inflate(-40, -40)
```

Kód 6.1: Ukázka třídy NPC

NPC jsou do hry přidávána na základě objektové vrstvy *Entities* v mapě vytvořené pomocí aplikace Tiled. Tím je zajištěno, že jejich pozici lze snadno měnit bez nutnosti zásahu do zdrojového kódu.



Obrázek 6.1: Ukázka NPC a hráče

6.2 DIALOG S NPC

Dialogový systém umožňuje hráči komunikovat s NPC pomocí klávesnice. Interakce je aktivována stiskem klávesy E v blízkosti NPC.

Pro správu dialogů byla vytvořena samostatná třída `DialogSystem`, která zajišťuje:

- spuštění dialogu,
- přechod mezi jednotlivými větami,
- dočasné zablokování pohybu hráče,
- zobrazení dialogového okna.

Dialogy jsou definovány odděleně od logiky hry ve slovníku `NPC_DIALOGUES`, kde je každému NPC přiřazen seznam vět:

```
1 NPC_DIALOGUES = {  
2     "villager_1": [  
3         "Temnota se šíří lesem...",  
4         "Najdi tři části klíče a vrať se ke mně."  
5     ]  
6 }
```

Kód 6.2: Ukázka slovníku NPC

Samotné zobrazení dialogu je realizováno pomocí knihovny `pygame_gui`, která umožňuje vykreslit textové okno nad herní scénou:

```
1 self.dialog_box = pygame_gui.elements.UITextBox(  
2     text,  
3     pygame.Rect(420, 540, 440, 100),  
4     self.manager  
5 )
```

Kód 6.3: Vytvoření dialogového textového boxu

ZÁVĚR

Cílem projektu bylo vytvořit funkční 2D hru v jazyce Python s využitím knihovny Pygame, rozšířenou o modernější prvky jako jsou grafické uživatelské rozhraní, dialogový systém a práce s mapami vytvořenými v externím editoru.

Během vývoje se podařilo implementovat herní svět vytvořený v aplikaci Tiled, který je ve hře načítán pomocí knihovny pytmx. Díky rozdělení mapy do vrstev je možné jednoduše oddělit vizuální část scény od kolizí a herních entit. Významnou součástí hry jsou také NPC postavy, které umožňují interakci s hráčem prostřednictvím dialogů a podporují jednoduché úkoly (např. sběr částí klíče).

Výsledná aplikace ukazuje praktické využití externích nástrojů a knihoven, které urychlují vývoj a zvyšují přehlednost projektu. Zároveň je projekt dobře připraven pro další rozšíření, například o nové mapy, více typů NPC, další úkoly, nebo rozšíření síťového režimu.

LITERATURA

Pygame Community Edition Documentation [online]. [cit. 2026-01-05]. Dostupné z: <https://pyga.me/>

Pygame Documentation [online]. [cit. 2026-01-05]. Dostupné z: <https://www.pygame.org/docs/>

Socket.IO Documentation [online]. [cit. 2026-01-05]. Dostupné z: <https://socket.io/>

python-socketio (GitHub repozitář) [online]. [cit. 2026-01-05]. Dostupné z: <https://github.com/miguelgrinberg/python-socketio>

Tiled Map Editor – Official Website [online]. [cit. 2026-01-05]. Dostupné z: <https://www.mapeditor.org/>

pytmx (GitHub repozitář) [online]. [cit. 2026-01-05]. Dostupné z: <https://github.com/bitcraft/pytmx>